

Security Audit

MarketDAO Updates (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	13
Audit Resources	13
Dependencies	13
Severity Definitions	14
Status Definitions	15
Audit Findings	16
Centralisation	44
Conclusion	45
Our Methodology	46
Disclaimers	48
About Hashlock	49

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The MarketDAO team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

MarketDAO is a governance framework that brings market forces to bear on group decisions. The key innovation is a system where voting rights can be freely bought and sold during elections, allowing market forces to influence governance outcomes.

Project Name: MarketDAO

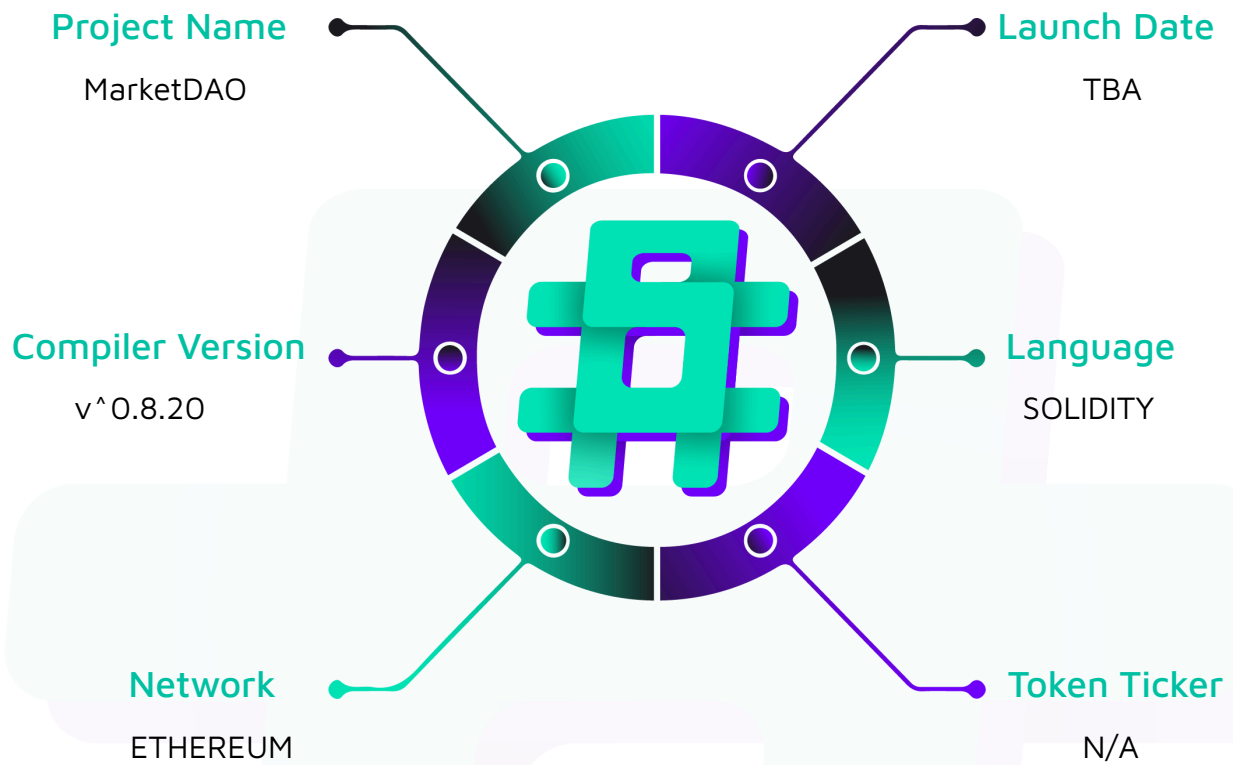
Project Type: DeFi

Compiler Version: ^0.8.20

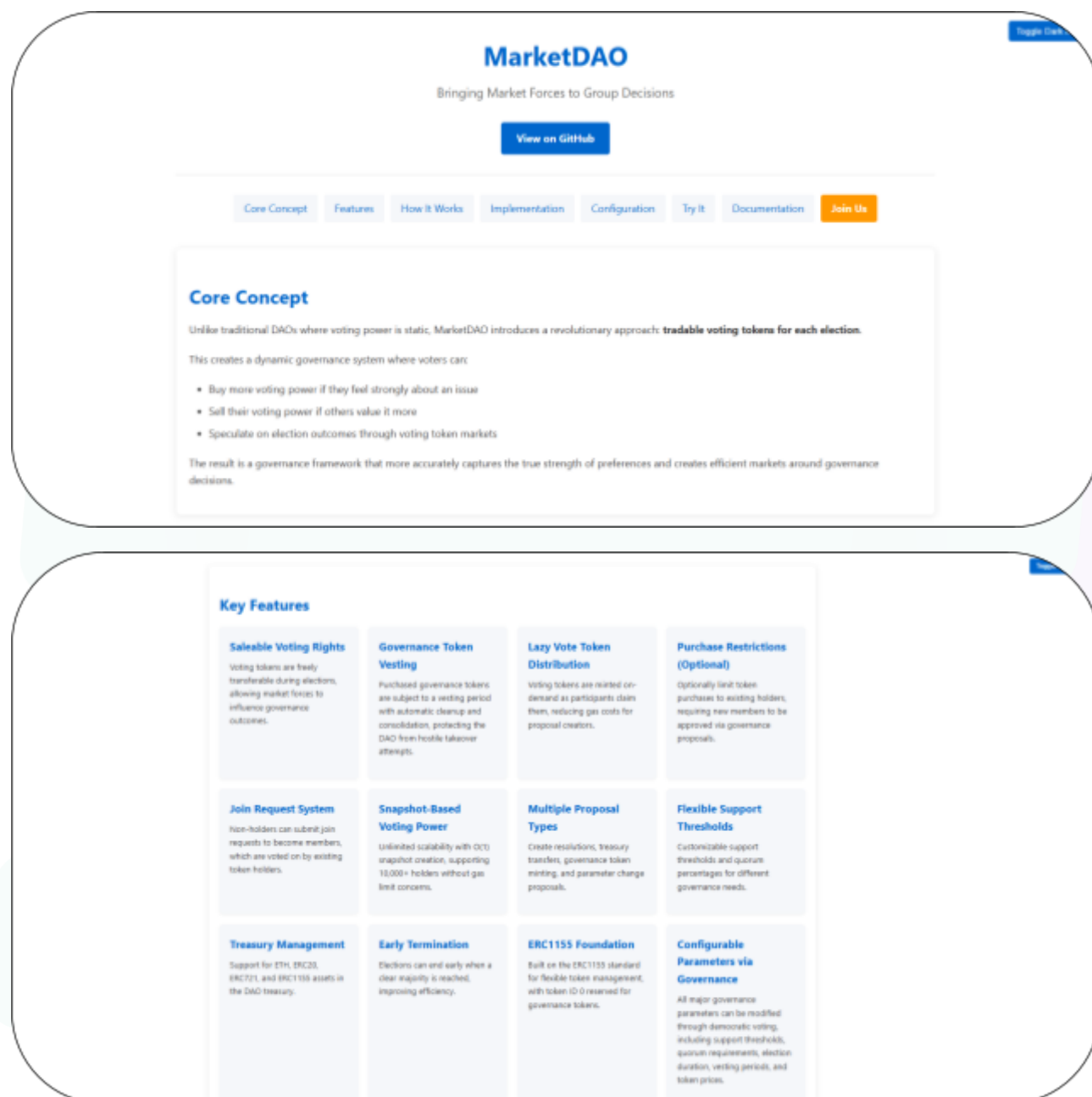
Website: <https://marketdao.dev>

Logo:

The logo for MarketDAO, featuring the text "MarketDAO" in a bold, blue, sans-serif font, enclosed within a white rounded rectangle with a thin black border.

Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the MarketDAO project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	MarketDAO Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	February, 2026
Contract 1	MarketDAO.sol
Contract 2	Proposal.sol
Contract 3	ProposalFactory.sol
Contract 4	ProposalTypes.sol
Contract 5	DistributionRedemption.sol
Contract 6	GenericProposal.sol
Audited GitHub Commit Hash	d511fed4daca43629d5cb536a317b114abcc591e
Fix Review GitHub Commit Hash	0a7b7cead6996e7e4f820f47c19957d9e7000c1e

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

5 High severity vulnerabilities

3 Medium severity vulnerabilities

2 Low severity vulnerabilities

4 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
MarketDAO.sol - Allows the DAO to: - Hold and manage treasury assets (ETH, ERC20, ERC721, ERC1155) - Mint and track governance tokens (ID=0)• Manage vesting schedules per holder - Track governance token holders - Lock and unlock treasury funds for active proposals - Register and deregister active proposals via the factory - Register vote addresses per proposal - Execute treasury transfers on behalf of active proposals - Allow direct token purchases via purchaseTokens - Manage distribution locks and governance locks - Enforce transfer restrictions on unvested and locked tokens	Contract achieves this functionality.
Proposal.sol - Allows the proposal lifecycle to: - Accept support from governance token holders - Allow support removal before the election trigger - Automatically trigger elections when the support threshold is met	Contract achieves this functionality.

- Lazily mint voting tokens to eligible holders during elections - Allow early termination when a strict majority is reached - Execute proposals after the election period ends - Fail proposals that do not meet quorum or majority - Lock and release governance tokens for support and voting - Allow post-resolution lock release by users	
ProposalFactory.sol - Allows the factory to: - Deploy GenericProposal clones for governance token holders - Deploy DistributionProposal clones for governance token holders - Allow non-holders to create join request proposals (mint 1 token to themselves) - Register created proposals as active in the DAO - Track all created proposals by sequential index	Contract achieves this functionality.
ProposalTypes.sol - Allows DistributionProposal to: - Initialize with a target asset and amount per governance token - Validate treasury has sufficient available balance at creation - Lock treasury funds and deploy a	Contract achieves this functionality.

DistributionRedemption contract on the election trigger - Allow governance token holders to register for distribution - Transfer funds to the redemption contract on execution - Unlock treasury funds on proposal failure Deprecated contracts (ResolutionProposal, TreasuryProposal, MintProposal, ParameterProposal) revert immediately on initialization	
DistributionRedemption.sol - Allows the redemption contract to: - Accept claimant registrations from the proposal contract - Lock governance tokens in the DAO on registration - Accept the funding snapshot from the proposal after execution - Allow registered users to claim their pro-rata share of the distribution pool - Unlock governance tokens on claim - Allow registered users to release their governance token lock after distribution ends - Provide view functions for claimable amounts and registration status	Contract achieves this functionality.
GenericProposal.sol - Allows GenericProposal to: - Initialize with an arbitrary target address, ETH value, and calldata	Contract achieves this functionality.

- | | |
|---|--|
| <ul style="list-style-type: none">- Automatically detect and validate DAO treasury operations at initialization- Lock treasury funds at the election trigger for detected treasury operations- Execute arbitrary calls to any contract on proposal passage- Unlock treasury funds and clear active proposal status on execution or failure- Support resolution proposals via empty calldata on the DAO target | |
|---|--|

Code Quality

This audit scope involves the smart contracts of the MarketDAO project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the MarketDAO project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from OpenZeppelin libraries, the contracts interact with each other through a series of internal cross-contract calls. GenericProposal is designed to optionally execute calls to arbitrary external contracts at governance discretion, but the protocol itself has no hardcoded external protocol dependencies.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] MarketDAO#setActiveRedemptionContract - Concurrent distribution proposals overwrite each other's redemption contract

Description

`activeRedemptionContract` is a single storage slot in MarketDAO. When a `DistributionProposal` triggers an election, it deploys a `DistributionRedemption` contract and registers it via `dao.setActiveRedemptionContract()`. If a second `DistributionProposal` triggers its election while the first is still active, it overwrites this slot with its own redemption contract.

Vulnerability Details

`DistributionRedemption` calls back into the DAO for lock management:

```
dao.lockForDistribution(user, governanceTokenBalance);  
  
dao.unlockForDistribution(user);
```

Both of these require:

```
require(msg.sender == activeRedemptionContract, "Only active redemption contract");
```

Impact

Complete loss of user funds locked in the first `DistributionRedemption` contract. Governance tokens of all registered users are permanently locked with no recovery path.

Recommendation

We recommend adding a check in `DistributionProposal._lockFunds()` or in `dao.setActiveRedemptionContract()` that reverts if `activeRedemptionContract !=`

`address(0)`, enforcing that only one distribution proposal can be in the election phase at any time.

Status

Resolved.



[H-02] DistributionProposal#registerForDistribution - Double registration via governance token transfer

Description

`registerForDistribution()` function records a user's share using their current live `vestedBalance` and immediately locks those tokens via `distributionLock`. However, a user can transfer their tokens to another address before registering, allowing the same tokens to be registered under multiple addresses.

Vulnerability Details

```
function registerForDistribution() external {  
  
    uint256 vestedBal = dao.vestedBalance(msg.sender); // live balance  
  
    redemptionContract.registerClaimant(msg.sender, vestedBal);  
  
}
```

The lock only applies after registration. The attack is:

1. Alice holds 100 governance tokens
2. Alice transfers 100 tokens to Bob
3. Bob registers -> Bob's share = 100, `distributionLock[Bob] = 100`
4. Bob transfers tokens back to Alice (Bob is now locked, but Alice received the tokens)
5. Alice registers -> Alice's share = 100, `distributionLock[Alice] = 100`

The same 100 tokens generated 200 registered shares, inflating both users' pro-rata claims at the expense of legitimate registrants. Every other voting function uses `vestedBalanceAt(electionStart)` specifically to prevent this class of manipulation.

Impact

Attackers can inflate their share of any distribution at the direct expense of honest participants. The pool is fixed, but `totalRegisteredGovernanceTokens` is artificially inflated, diluting everyone else's claim.

Recommendation

We recommend replacing `dao.vestedBalance(msg.sender)` with `dao.vestedBalanceAt(msg.sender, electionStart)` in `registerForDistribution()`, consistent with how voting power is calculated everywhere else in the codebase.

Status

Resolved.

[H-03] MarketDAO#purchaseTokens and Proposal#claimVotingTokens - Token purchases during active elections inflate voting power beyond snapshotTotalVotes

Description

vestedBalanceAt is used to calculate voting power at claim time and also reads the holder's current token balance, but applies vesting locks as they existed at electionStart. When vestingPeriod = 0, there are no vesting locks, so a buyer who purchases tokens after an election starts receives voting power equal to their full current balance, which includes tokens that didn't exist when the snapshot was taken.

Vulnerability Details

snapshotTotalVotes is recorded at the election trigger:

```
snapshotTotalVotes = dao.getTotalVestedSupply();

// = tokenSupply[0] - totalUnvestedGovernanceTokens

// recorded once, never updated
```

claimVotingTokens uses vestedBalanceAt, which reads the current balance:

```
function vestedBalanceAt(address holder, uint256 blockNumber) public view returns
(uint256) {

    uint256 locked = 0;

    for (uint256 i = 0; i < schedules.length; i++) {

        if (blockNumber < schedules[i].unlockBlock) {

            locked += schedules[i].amount;

        }

    }

    uint256 currentBalance = balanceOf(holder, GOVERNANCE_TOKEN_ID); // current, not
snapshot

    if (locked >= currentBalance) return 0;

    return currentBalance - locked; // no locks when vestingPeriod = 0
```



```
}
```

`purchaseTokens` with `mintToPurchase = false` (default) mints new tokens, increasing `tokenSupply[0]` but not `snapshotTotalVotes`:

```
_mint(msg.sender, GOVERNANCE_TOKEN_ID, tokenAmount, "");

tokenSupply[GOVERNANCE_TOKEN_ID] += tokenAmount; // supply grows post-snapshot

// snapshotTotalVotes is unchanged
```

Full attack with `vestingPeriod = 0`:

1. Election starts, `snapshotTotalVotes = 1000`, attacker holds 100 tokens
2. Attacker calls `purchaseTokens()` buying 5000 new tokens -> `tokenSupply[0] = 6000`
3. `claimVotingTokens()`: `vestedBalanceAt(attacker, electionStart) = 5100 - 0 = 5100`
4. Attacker receives 5100 voting tokens against a snapshot of 1000
5. `majorityThreshold = 1000 / 2 + 1 = 501`
6. Attacker transfers 501 voting tokens to YES address
7. `checkEarlyTermination: yesVotes(501) >= majorityThreshold(501) AND quorumRequired` easily met -> proposal executes immediately

Any proposal passes instantly. Attackers can drain the entire treasury in a single election.

Note: `vestingPeriod > 0` fully mitigates this. New purchases create a schedule with `unlockBlock = block.number + vestingPeriod`. Since `electionStart < unlockBlock`, the new tokens ARE counted as locked at `electionStart`, zeroing out the gained voting power. The bug is specific to `vestingPeriod = 0`, which is a valid and default deployment configuration (`flags = 0`).

Impact

Complete governance takeover with sufficient capital. Attackers force through any proposal - treasury drain, minting tokens to themselves, any parameter change - by buying tokens during an active election and overwhelming the snapshot-based majority threshold.

Recommendation

We recommend replacing `currentBalance` in `vestedBalanceAt` with a per-address balance snapshot taken at `electionStart`. The simplest approach is to snapshot `balanceOf(holder, GOVERNANCE_TOKEN_ID)` for each user when they claim voting tokens, or to record a per-address snapshot at election trigger. Alternatively, enforce that `purchaseTokens` is disabled during active elections.

Status

Resolved.

[H-04] GenericProposal#_execute - Permanently locked treasury funds when execution call fails on a passed proposal

Description

When a proposal passes (YES wins, quorum met) but the execution call permanently reverts, treasury funds are locked forever. Neither `execute()` nor `failProposal()` can recover them and the proposal remains stuck in a state where it passed but cannot execute.

Vulnerability Details

`super._execute()` sets `executed = true` before the external call. If the call fails, the entire transaction reverts, rolling `executed` back to `false`:

```
// GenericProposal._execute

function _execute() internal override {

    super._execute(); // executed = true - but rolled back if tx reverts

    if (data.length > 0 || target != address(dao)) {

        (bool success, bytes memory result) = target.call{value: value}(data);

        if (!success) {

            revert(...); // entire tx reverts including executed = true

        }

    }

    _unlockFunds(); // never reached

    dao.clearActiveProposal(); // never reached

}
```

With `executed = false` and the election over, both recovery paths fail:

```
// execute() → _execute() → target.call fails → revert → loops forever
```

```
// failProposal():

bool quorumNotMet = totalVotes < quorumRequired; // false - quorum was met

bool rejected = yesVotes <= noVotes;           // false - YES won

require(quorumNotMet || rejected);              // false || false → REVERTS
```

There is no third path. `_tryReleaseLockedProposals` also cannot help because `clearActiveProposal()` was never called - `activeProposals[proposal]` is still true.

Realistic scenarios that cause a permanent execution failure:

- ETH transfer to a contract without `receive()`
- `dao.transferETH` does `require(success)` on the inner call
- ERC20 transfer to a blacklisted address (USDC, USDT have on-chain blacklists)
- Mint proposal where `allowMinting()` flag was flipped off by a concurrent parameter proposal before execution
- External call to a self-destructed contract

Impact

Treasury funds locked in the proposal are permanently unrecoverable. The DAO has no admin escape hatch, no upgrade mechanism and no fallback path once a proposal is in this state.

Recommendation

We recommend moving `_unlockFunds()` and `dao.clearActiveProposal()` into a finally-equivalent pattern - specifically, restructure `_execute` to set a local success flag and always run cleanup regardless of call outcome. One clean approach: catch the failure, mark the proposal as failed (not executed), call `_unlockFunds` and call `clearActiveProposal` before reverting with the original error.

Status

Resolved.

[H-05] DistributionProposal#registerForDistribution - Post-execution registration dilutes legitimate claimants and drains distribution pool

Description

registerForDistribution has no check that the proposal hasn't executed yet. Combined with activeRedemptionContract never being cleared during _execute, an attacker can register after the pool is funded, inflate totalRegisteredGovernanceTokens and drain the majority of funds meant for legitimate registrants.

Vulnerability Details

DistributionProposal._execute() clears the treasury lock and active proposal status but never clears activeRedemptionContract:

```
function _execute() internal override {

    super._execute();

    dao.transferETH(payable(address(redemptionContract)), totalAmount);

    redemptionContract.markPoolFunded(); // totalPoolBalance fixed here

    dao.unlockFunds();

    dao.clearActiveProposal();

    // clearActiveRedemptionContract() never called

}
```

registerForDistribution has no executed check:

```
function registerForDistribution() external {

    if (!electionTriggered) revert ElectionNotTriggered();

    if (address(redemptionContract) == address(0)) revert RedemptionNotDeployed();

    // no require(!executed) check

    uint256 vestedBal = dao.vestedBalance(msg.sender);

    redemptionContract.registerClaimant(msg.sender, vestedBal);

}
```

`registerClaimant` succeeds post-execution because `activeRedemptionContract` is still set:

```
// In MarketDAO.lockForDistribution:

require(msg.sender == activeRedemptionContract); // still passes post-execution
```

Pro-rata claim calculation: $\text{claimAmount} = (\text{userShares} * \text{totalPoolBalance}) / \text{totalRegisteredGovernanceTokens}$

`totalPoolBalance` is fixed at execution time. `totalRegisteredGovernanceTokens` grows unboundedly post-execution. Every new registration directly dilutes all prior registrants.

Concrete attack with pool = 1000 ETH:

1. 10 legitimate users register pre-execution, each with 100 tokens -> `totalRegisteredGovernanceTokens` = 1000, each expects 100 ETH
2. Proposal executes, `totalPoolBalance` = 1000 ETH
3. Attacker registers post-execution with 9000 tokens -> `totalRegisteredGovernanceTokens` = 10000
4. Legitimate users each receive: $(100 * 1000) / 10000 = 10$ ETH
5. Attacker receives: $(9000 * 1000) / 10000 = 900$ ETH

Impact

An attacker with sufficient governance tokens can steal the vast majority of any distribution pool after it executes. Legitimate registrants lose funds proportional to how many tokens the attacker holds. The larger the attacker's token balance relative to legitimate registrants, the more they steal.

Recommendation

We recommend adding `require(!executed, "Distribution already executed")` at the start of `registerForDistribution` and calling `dao.clearActiveRedemptionContract()` at the end of `_execute` to close the window completely.

Status

Resolved.

Medium

[M-01] DistributionRedemption#releaseLock - Calling `releaseLock` does not forfeit the claim, allowing users to both unlock governance tokens and claim distribution funds

Description

`releaseLock` is designed as a "I don't want to participate, free my governance tokens" path. However, it neither sets `hasClaimed = true` nor zeroes out `registeredBalance`, meaning a user can call `releaseLock` to unlock their governance tokens and then subsequently still call `claim` to collect their distribution share. The two paths are not mutually exclusive as intended.

Vulnerability Details

`releaseLock` only calls `unlockForDistribution` but leaves claim state completely intact:

```
function releaseLock() external {  
  
    if (registeredBalance[msg.sender] == 0) revert NotRegistered();  
  
    bool proposalEnded = IDistributionProposal(proposal).executed() ||  
                        !dao.activeProposals(proposal);  
    if (!proposalEnded) revert DistributionStillActive();  
  
    if (hasClaimed[msg.sender]) return; // already claimed → no-op  
  
    dao.unlockForDistribution(msg.sender); // governance tokens freed  
  
    // registeredBalance[msg.sender] unchanged - still > 0  
    // hasClaimed[msg.sender] unchanged - still false  
}
```

After `releaseLock`, claim passes all checks:

```
function claim() external nonReentrant {  
  
    if (registeredBalance[msg.sender] == 0) revert; // still set → passes  
  
    if (hasClaimed[msg.sender]) revert;           // still false → passes  
  
    if (!poolFunded) revert;                      // funded → passes  
  
    // calculates and pays out claimAmount  
  
    hasClaimed[msg.sender] = true;  
  
    dao.unlockForDistribution(msg.sender); // already 0, no-op  
  
    // transfer funds to user  
  
}
```

The user ends up with both freed governance tokens AND their distribution share - the intended trade-off of `releaseLock` is completely bypassed.

Impact

Users can have it both ways - unlock their governance tokens immediately via `releaseLock` to use them elsewhere in governance and later return to claim their full distribution share. This breaks the intended mutual exclusivity of the two paths confirmed by the dev.

Recommendation

We recommend setting `hasClaimed[msg.sender] = true` and zeroing out `registeredBalance[msg.sender]` inside `releaseLock` before calling `unlockForDistribution`, permanently forfeiting the claim when a user chooses to release their lock.

Status

Resolved.

[M-02] Proposal#addSupport - Same tokens can satisfy support threshold across multiple proposals simultaneously

Description

addSupport validates the caller's available balance using vestedBalance, which only subtracts vesting schedule locks from the raw token balance. It completely ignores governanceLock, meaning the same governance tokens can be used to push multiple proposals past the support threshold simultaneously.

Vulnerability Details

```
function addSupport(uint256 amount) external onlyBeforeElection {
    uint256 availableBalance = dao.vestedBalance(msg.sender); // governanceLock not
    subtracted

    require(availableBalance >= amount, "Insufficient vested governance tokens");

    require(support[msg.sender] + amount <= availableBalance, ...);

    dao.addGovernanceLock(msg.sender, amount); // locked, but invisible to next
    proposal's check

    support[msg.sender] += amount;

    supportTotal += amount;
}
```

vestedBalance vs transferableBalance:

```
function vestedBalance(address holder) public view returns (uint256) {
    return balanceOf(holder, GOVERNANCE_TOKEN_ID) - vestingLocked; // governanceLock
    ignored
}

function transferableBalance(address holder) public view returns (uint256) {
    return vestedBalance(holder) - distributionLock[holder] - governanceLock[holder];
}
```

```
}
```

support is a per-proposal mapping on each clone, so on every new proposal, it starts at 0. This means the accumulated governanceLock from previous proposals is never reflected in the balance check on the next one.

Concrete scenario with supportThreshold = 20% and attacker controlling exactly 20% of vested supply:

1. Attacker calls addSupport(threshold) on Proposal A -> governanceLock += threshold, election triggers
2. On Proposal B: support[attacker] = 0, vestedBalance still returns full balance -> threshold satisfied again, election triggers
3. Repeat for N proposals - the same tokens satisfy the threshold every time

Impact

An attacker holding just above the support threshold can force arbitrarily many simultaneous elections. Every honest holder must actively monitor and vote NO on all of them or risk a malicious proposal passing. This drains honest participants of gas and attention and makes it statistically easier for a malicious proposal to slip through.

Recommendation

We recommend replacing `dao.vestedBalance(msg.sender)` with `dao.transferableBalance(msg.sender)` in `addSupport`, which already accounts for `governanceLock` and is consistent with how transfer validation works elsewhere in the codebase.

Status

Acknowledged.

[M-03] Proposal#addSupport - Zero vested supply collapses support threshold to zero, allowing anyone to grief proposals into failed elections

Description

When `getTotalVestedSupply()` returns 0, the support threshold becomes 0. A call to `addSupport(0)` from any address - including one with no governance tokens - passes all checks and immediately triggers an election that can be immediately failed.

Vulnerability Details

Threshold computation:

```
uint256 threshold = (dao.getTotalVestedSupply() * dao.supportThreshold()) / 10000;

// If getTotalVestedSupply() == 0 → threshold = 0

return supportTotal >= threshold; // 0 >= 0 → always true
```

`addSupport(0)` passes all validation when the caller has zero vested balance:

```
uint256 availableBalance = dao.vestedBalance(msg.sender); // = 0

require(availableBalance >= amount); // 0 >= 0 ✓

require(support[msg.sender] + amount <= availableBalance); // 0 <= 0 ✓

dao.addGovernanceLock(msg.sender, 0); // no-op

supportTotal += 0;

canTriggerElection() → true → _triggerElection()
```

The triggered election gets `snapshotTotalVotes = 0`, so `quorumRequired = 0`. No one can claim voting tokens (`vestedBalanceAt` returns 0 for everyone). `failProposal` is immediately callable because `yesVotes <= noVotes` evaluates as `0 <= 0 = true`.

This is exploitable in any DAO that launches via `purchaseTokens` with `vestingPeriod > 0` and no initially vested holders - a valid and documented deployment configuration. The window exists from deployment until the first vesting schedules expire.

Impact

Any address can immediately grief any pending proposal into a forced failed election before legitimate supporters can act. No funds are at risk but all active proposals can be systematically destroyed, fully blocking governance during the vesting window.

Recommendation

We recommend adding `require(amount > 0)` at the start of `addSupport` and adding `require(getTotalVestedSupply() > 0)` inside `canTriggerElection` before computing the threshold.

Status

Resolved.

Low

[L-01] MarketDAO - Governance parameter changes immediately affect all active elections

Description

When a GenericProposal changing quorumPercentage, electionDuration or supportThreshold executes, the new values take effect instantly across the entire DAO - including all currently active elections that began under the old parameters.

Vulnerability Details

Parameters like electionDuration are read at execution time, not snapshotted at election start:

```
modifier onlyDuringElection() {  
  
    require(  
  
        block.number < electionStart + dao.electionDuration(), // reads live value  
  
        "Not during election period"  
  
    );  
  
}
```

A passed proposal that doubles the election duration immediately extends all ongoing elections. One that halves quorumPercentage immediately lowers the bar for concurrent proposals to pass. There is no isolation between a parameter change and other in-flight governance.

Impact

Parameter change proposals can retroactively alter the rules of concurrent elections. This could be used to strategically extend elections that are losing or lower quorum requirements to push through a concurrent proposal that would otherwise fail. The severity is somewhat mitigated by the fact that the parameter change itself must pass a DAO vote.

Recommendation

We recommend snapshotting governance parameters (electionDuration, quorumPercentage) at election trigger time and storing them per-proposal, so each election runs to completion under the rules that were in place when it started.

Status

Acknowledged.

[L-02] MarketDAO#function - Missing zero-address validation on treasury transfer functions

Description

All four treasury transfer functions - `transferETH`, `transferERC20`, `transferERC721`, `transferERC1155` - have no zero-address check on the recipient parameter. A passed governance proposal with `recipient = address(0)` would permanently burn treasury assets with no recovery path.

Vulnerability Details

```
function transferETH(address payable recipient, uint256 amount) external {  
    require(activeProposals[msg.sender], "Only active proposal can transfer");  
  
    (bool success, ) = recipient.call{value: amount}("");  
  
    require(success, "ETH transfer failed");  
}
```

`address(0).call{value: amount}("")` succeeds on EVM - ETH is silently and permanently burned. The same applies to ERC20 and ERC1155 transfers where `safeTransfer` to `address(0)` may succeed depending on the token implementation. This does not require malicious intent - a proposal with a misconfigured recipient field that passes governance would permanently destroy treasury funds with no recovery.

Impact

Permanent, unrecoverable loss of treasury assets from a legitimate but misconfigured governance proposal.

Recommendation

We recommend adding `require(recipient != address(0), "Invalid recipient")` to all four treasury transfer functions.

Status

Acknowledged.

QA

[Q-01] GenericProposal#_execute - Redundant executed = true assignment

Description

executed = true is set twice in GenericProposal._execute() - once in super._execute() and once redundantly in the override.

Vulnerability Details

```
function _execute() internal override {  
  
    super._execute(); // sets executed = true ← first assignment  
  
    if (data.length > 0 || target != address(dao)) {  
        (bool success, bytes memory result) = target.call{value: value}(data);  
        if (!success) { revert(...); }  
    }  
  
    executed = true; // ← redundant second assignment  
    _unlockFunds();  
    dao.clearActiveProposal();  
}
```

The second assignment is unreachable in any meaningful way - if super._execute() sets it, and the external call succeeds; it was already true. If the external call fails, the revert rolls back the entire transaction, including the first assignment, and execution never reaches the second line anyway.

Impact

No security impact. Creates misleading code that implies `execution` is set after the external call, which could confuse future developers into thinking CEI pattern is not followed here, leading to incorrect modifications.

Recommendation

We recommend removing the redundant `executed = true` line from `GenericProposal._execute()` and adding a NatSpec comment on `super._execute()` clarifying that it sets `executed = true` as the first action.

Status

Resolved.

[Q-02] ProposalFactory#function - Inconsistent description validation between createProposal and createDistributionProposal

Description

createProposal enforces a non-empty description via `require(bytes(description).length > 0, "Description required")`. createDistributionProposal has no equivalent check, allowing distribution proposals to be created with empty descriptions.

Vulnerability Details

```
// createProposal - validates description
require(bytes(description).length > 0, "Description required"); // ✓

// createDistributionProposal - no validation
function createDistributionProposal(
    string memory description, // can be empty string ""
    address token,
    uint256 tokenId,
    uint256 amountPerToken
) external onlyTokenHolder returns (DistributionProposal) {
    address clone = Clones.clone(distributionImpl);
    DistributionProposal(clone).initialize(
        dao,
        description, // empty string passes through unchecked
        ...
    );
}
```

__Proposal_init in Proposal.sol performs no description validation either, so an empty string propagates all the way through to storage.

Impact

Distribution proposals with empty descriptions can be created and voted on. Governance participants have no human-readable context for what they are voting on. Combined with the absence of proposal creation events already noted, this makes governance tracking and accountability significantly harder.

Recommendation

We recommend adding `require(bytes(description).length > 0, "Description required")` at the start of `createDistributionProposal`, consistent with `createProposal`.

Status

Acknowledged.

[Q-03] DistributionRedemption#releaseLock - Duplicate LockReleased events emitted on repeated calls

Description

releaseLock can be called infinitely after the first successful call, emitting a LockReleased event each time with no actual state change. registeredBalance is never zeroed, and hasClaimed is never set to true inside releaseLock, so all entry checks pass on every subsequent call.

Vulnerability Details

After the first successful `releaseLock`:

- registeredBalance[user] remains unchanged - NotRegistered check always passes
- hasClaimed[user] remains false - early return never triggers
- distributionLock[user] = 0 - unlockForDistribution is a no-op
- LockReleased event emitted again with zero actual effect

```
function releaseLock() external {
    if (registeredBalance[msg.sender] == 0) revert NotRegistered(); // never zeroed → passes

    bool proposalEnded = ...;

    if (!proposalEnded) revert DistributionStillActive(); // still true → passes

    if (hasClaimed[msg.sender]) { return; } // false → doesn't return

    dao.unlockForDistribution(msg.sender); // already 0, no-op

    emit LockReleased(msg.sender); // duplicate event ← problem
}
```


Any off-chain indexer or frontend tracking `LockReleased` events to determine lock status will see repeated emissions for the same user, making it impossible to reliably determine the current lock state from events alone.

Impact

Off-chain systems relying on `LockReleased` events for state tracking receive corrupted data. Multiple emissions for the same user with no corresponding state change makes event-based indexing unreliable.

Recommendation

We recommend zeroing `registeredBalance[msg.sender]` at the end of `releaseLock` to prevent repeated calls, which also naturally fixes the `isRegistered` view function returning stale data for users who have released their locks.

Status

Resolved.

[Q-04] ProposalFactory + DistributionProposal - Missing events across proposal creation and distribution lifecycle

Description

Critical governance events are missing across two contracts. ProposalFactory emits no events on proposal creation, and DistributionProposal emits no events on execution or failure.

Vulnerability Details

```
// ProposalFactory - no event on creation

proposals[proposalCount++] = clone;

return GenericProposal(clone); // no event emitted


// DistributionProposal - no event on failure

function _unlockFunds() internal override {

    dao.unlockFunds(); // silent - no event

}
```

Off-chain infrastructure has no reliable way to discover proposals without scanning every transaction to the factory. Users who registered for a failed distribution have no on-chain signal to know they need to call releaseLock() to free their governance tokens.

Impact

Frontends and indexers cannot reliably track governance activity. Registered distribution users remain governance-locked with no notification after a failed distribution.

Recommendation

We recommend emitting ProposalCreated(uint256 indexed id, address indexed proposal, address indexed proposer, string description) in both createProposal and

`createDistributionProposal`, and emitting `DistributionExecuted` and `DistributionCancelled` events inside `_execute` and `_unlockFunds` in `DistributionProposal`.

Status

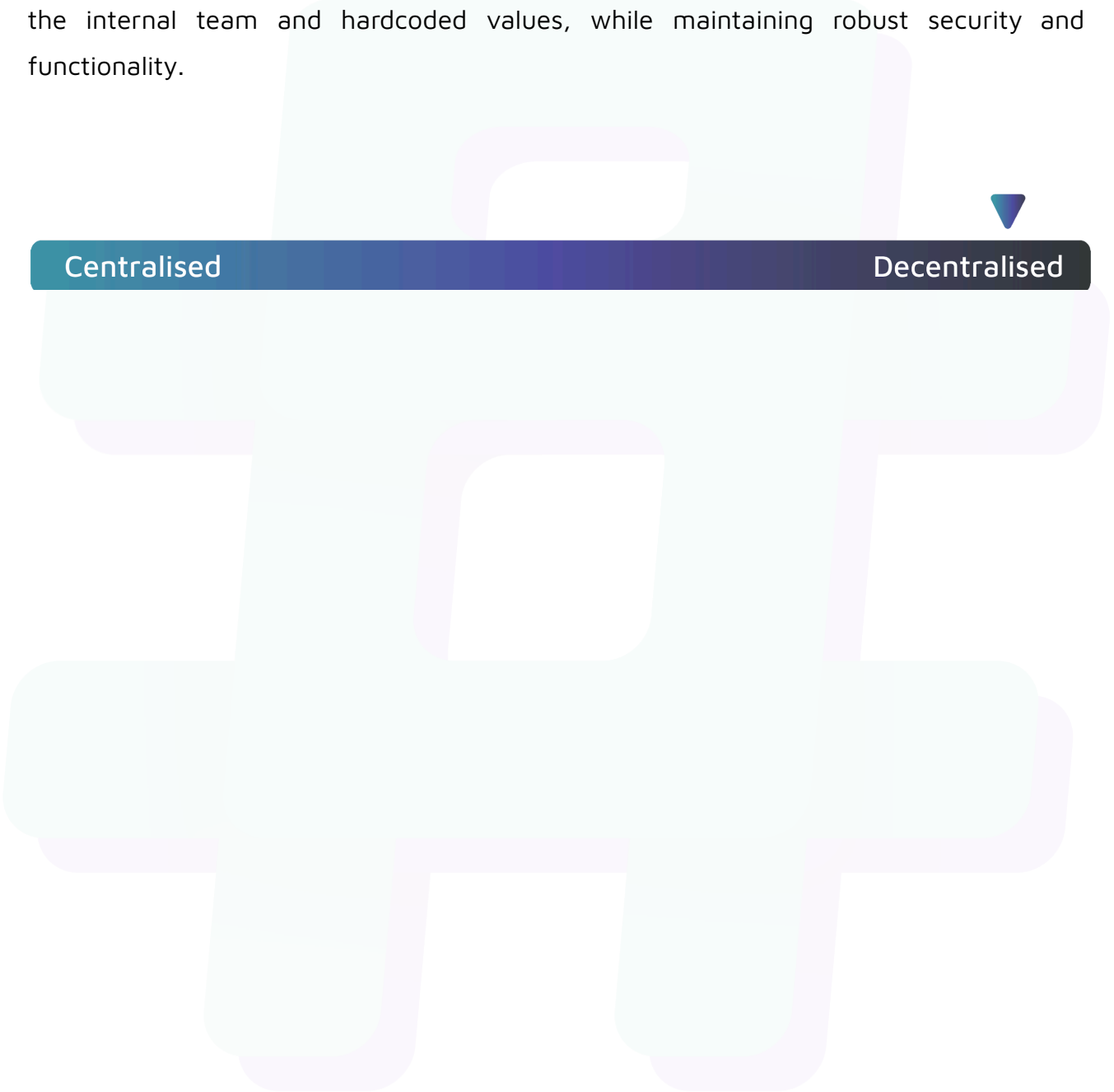
Resolved.



Centralisation

The MarketDAO project values decentralisation alongside security and utility.

The protocol's functions are designed to operate independently, minimising reliance on the internal team and hardcoded values, while maintaining robust security and functionality.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the MarketDAO project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.

